

Fundamentos de Data Binding

```
TS app.ts Projeto02_DataBinding\...  </> app.html Projeto02_DataBinding\...  TS app.ts Projeto02.1_DataBinding\...  </> app.html Projeto02.1_DataBinding\...

Projeto02_DataBinding > src > app > TS app.ts > App
1  import { Component, signal } from '@angular/core';    // Importa o signal do Angular para criar sinais
2
3  @Component({
4    selector: 'app-root',
5    standalone: true,
6    templateUrl: './app.html',
7    styleUrls: ['./app.css']
8  })
9  export class App {
10     // Criando um sinal numérico inicializado em 0
11     protected contador = signal(0);
12
13     // Método para incrementar o valor
14     incrementar() {
15       this.contador.update(valor => valor + 1);
16     }
17
18     // Método para decrementar o valor
19     decrementar() {
20       this.contador.update(valor => valor - 1);
21     }
22   }
```

```
TS app.ts Projeto02_DataBinding\...  </> app.html Projeto02_DataBinding\...  X  TS app.ts Projeto02.1_DataBinding\...  </> app.html Projeto02.1_DataBinding\...

Projeto02_DataBinding > src > app > </> app.html > main > button
Go to component
1  <main>
2    <h1>Projeto 02: Data Binding</h1>
3
4    <p>O valor atual é: <strong>{{ contador() }}</strong></p> <!--Interpolação, variável contador exibi dados no html-->
5
6    <button (click)="incrementar()">Aumentar</button>
7    <!--Event Binding, cada vez que o botão for clicado, a função incrementar será chamada-->
8    <button (click)="decrementar()">Diminuir</button>
9    <!--Event Binding, cada vez que o botão for clicado, a função decrementar será chamada-->
10
11    <hr>
12
13    <button [disabled]="contador() < 1" (click)="contador.set(0)">Zerar Contador</button>
14    <!--Property Binding, o botão será desabilitado se o contador for menor que 1-->
15  </main>
16
17  <!--
18    Interpolação: {{ variavel }}
19    Event Binding: (evento)="funcao()"
20    Property Binding: [propriedade]="valor"
21  -->
```

1. Interpolação {{ }}

A interpolação é a forma mais simples de exibir dados. Ela permite que uma variável definida no **TypeScript** (como o seu `signal`) seja renderizada dinamicamente no **HTML**.

- **No Código:** `{{ contador() }}`
- **Funcionamento:** Sempre que o valor da variável muda no código, o Angular atualiza o texto na tela automaticamente.

2. Event Binding (`evento`)

É a comunicação do **HTML para o TypeScript**. Ele captura uma interação do usuário (como um clique) e dispara uma lógica de programação.

- **No Código:** `(click)="incrementar()"`
- **Funcionamento:** Ao detectar o evento de clique no botão, o Angular executa a função correspondente dentro da classe no arquivo `.ts`.

3. Property Binding [`propriedade`]

É a comunicação que define atributos de elementos HTML baseando-se em lógica. Quando o valor da expressão muda, o atributo se atualiza instantaneamente.

- **No Código:** `[disabled]="contador() < 1"`
- **Funcionamento:** O botão é desabilitado ou habilitado em tempo real, sem necessidade de recarregar a página, dependendo do valor atual do contador.

Conceito	Sintaxe	Direção do Fluxo	Objetivo
Interpolação	<code>{{ }}</code>	TS → HTML	Exibir textos/valores.
Property Binding	<code>[]</code>	TS → HTML	Alterar atributos de tags.
Event Binding	<code>()</code>	HTML → TS	Executar funções/métodos.

Two-Way Binding	<code>[()]</code>	TS ↔ HTML	Sincronizar formulários.
-----------------	--------------------	-----------	--------------------------